Nil and Errors Handling

In **Go**, error handling is **very different from languages like Java, Python, or C#**.
Go **does not use exceptions as the primary mechanism**. Instead, it uses:

1. **error return values**
2. **nil checks**
3. **panic / recover (rare, for truly exceptional situations)**

Let's understand each clearly.

---

1. nil in Go

What is nil?

nil is the **zero value for reference types**.

It represents **absence of value**.

Types that can be nil

Type	Meaning of nil
pointer	points to nothing
slice	empty slice reference
map	uninitialized map
channel	no channel
function	no function
interface	no concrete value
error	no error

Example:

```
package main

import "fmt"


func main() {


   var p *int

   var s []int

   var m map[string]int

   var err error


   fmt.Println(p)   // nil

   fmt.Println(s)   // nil

   fmt.Println(m)   // nil

   fmt.Println(err) // nil

}
```

Output

```
<nil>

<nil>

map[]

<nil>
```

---

2. Error Handling in Go (Primary Mechanism)

Instead of exceptions, Go functions **return errors explicitly**.

Pattern

```
result, err := function()


if err != nil {
   handle error
}
```

Example

```
package main
import (
	"fmt"
	"errors"
)

func divide(a, b int) (int, error) {

	if b == 0 {
		return 0, errors.New("division by zero")
	}

	return a / b, nil
}
```

```
func main() {

	result, err := divide(10, 0)

	if err != nil {
		fmt.Println("Error:", err)
		return
	}

	fmt.Println("Result:", result)
}
```

Output

Error: division by zero

Key Idea:

nil error → success

non-nil error → failure

---

3. Creating Errors

Go provides the **errors package**.

Method 1: errors.New()

```
import "errors"
```

```
err := errors.New("something went wrong")
```

Method 2: fmt.Errorf()

```
err := fmt.Errorf("user %d not found", id)
```

Example:

```
return fmt.Errorf("file %s not found", filename)
```

---

4. Panic (Similar to Exceptions but Rare)

panic is used for **unrecoverable errors**.

Examples:

- programmer mistake
- corrupted state
- impossible situations

Example:

```
package main
import "fmt"

func test() {
	panic("something bad happened")
}

func main() {
	test()
```

```
	fmt.Println("This will not run")
}
```

Output

panic: something bad happened

---

5. Recover (Catch Panic)

recover() is used inside **defer** to catch panic.

Example:

```
package main
import "fmt"

func safe() {

	defer func() {
		if r := recover(); r != nil {
			fmt.Println("Recovered from:", r)
		}
	}()

	panic("something wrong")
}

func main() {
```

```
        safe()
        fmt.Println("Program continues")
}
```

Output

```
Recovered from: something wrong
Program continues
```

---

6. Go Error Handling Flow

Typical Go pattern:

```
function call
    ↓
check err != nil
    ↓
handle error
```

Example:

```
file, err := os.Open("data.txt")
if err != nil {
   log.Fatal(err)
}
```

---

7. nil Pointer Panic Example

If you dereference a nil pointer → panic.

Example:

```go
package main

import "fmt"


func main() {


	var p *int

	fmt.Println(*p) // panic

}
```

Output

panic: runtime error: invalid memory address

---

8. Best Practices in Go

✓ Use error returns (most common)

```go
func Do() (result, error)
```

✓ Check errors immediately

```go
if err != nil { return err }
```

✓ Use panic only for programmer errors

✓ Use recover only in frameworks or middleware

---

9. Error vs Panic vs Nil

Concept	Meaning	Usage
nil	absence of value	default for references

Concept	Meaning	Usage
error	expected failure	file not found
panic	unexpected failure	programmer bug
recover	catch panic	middleware

---

10. Real Production Example

```
func ReadConfig(path string) (*Config, error) {

	file, err := os.Open(path)
	if err != nil {
		return nil, err
	}

	defer file.Close()

	cfg := &Config{}
	err = json.NewDecoder(file).Decode(cfg)

	if err != nil {
		return nil, err
	}

	return cfg, nil
```

```
}
```

Usage:

```
cfg, err := ReadConfig("config.json")

if err != nil {
   log.Fatal(err)
}
```

---

11. Comparison with Java

Java	Go
try	explicit error return
catch	if err != nil
throw	return error
Exception	error interface
finally	defer

✅ Summary

Go Error Handling Model

_____

Normal failures → error return

No error → nil

Serious failures → panic

Catch panic → recover

```
package main

import "fmt"

func main() {
  defer func() {
    if err := recover(); err != nil {
      fmt.Println(err)
    }
  }()
  var p *int
  fmt.Println(*p) // panic
  fmt.Println("done")
}
```

Your program is **almost correct**. The reason "done" is **not printed** is because of how **panic and recover work in Go**.

When a panic occurs:

1. Normal execution **stops immediately**.
2. Deferred functions **execute**.
3. If recover() catches the panic, the function **returns**.
4. Execution **does NOT continue after the panic line**.

So your code behaves like this:

fmt.Println(*p) -> panic occurs

defer runs

recover catches panic

main() returns

Therefore "done" is never reached.

Why your code doesn't print "done"

Your code:

```
package main


import "fmt"


func main() {
	defer func() {
		if err := recover(); err != nil {
			fmt.Println(err)
		}
	}()


	var p *int
	fmt.Println(*p) // panic
	fmt.Println("done")
}
```

Execution flow:

main start

↓

fmt.Println(*p) → panic

↓

defer executes

recover() catches panic

↓

main returns

So "done" is skipped.

Correct Way to Print "done"

You must **move the risky code into another function** and recover there.

Fixed Code

```
package main


import "fmt"


func safe() {
	defer func() {
		if err := recover(); err != nil {
			fmt.Println("Recovered:", err)
		}
```

```
    }()

    var p *int
    fmt.Println(*p) // panic
}

func main() {

    safe()

    fmt.Println("done")
}
```

Output

```
Recovered: runtime error: invalid memory address or nil pointer dereference
done
```

Why this works

Execution flow:

```
main()
  ↓
safe()
  ↓
panic happens
```

↓

recover() handles it

↓

safe() returns normally

↓

main continues

↓

prints "done"

Important Rule in Go

Recover only stops panic inside the same function where defer runs.

If panic occurs:

function

↓

panic

↓

defer runs

↓

recover

↓

function returns

It **does not resume execution after the panic line**.

Visual Flow

Your program

main

↓

panic

↓

recover

↓

main exits

Correct program

main

↓

safe()

↓

panic

↓

recover

↓

return to main

↓

done printed

How panic propagates through the call stack

In **Go**, when a **panic occurs**, Go **unwinds the call stack** until one of two things happens:

1. A **recover() catches the panic**
2. The stack reaches the **top (main goroutine)** → program crashes

This process is called **panic propagation**.

---

1. What is Call Stack?

The **call stack** is the sequence of function calls.

Example:

main → A → B → C

If panic occurs in C, Go starts **unwinding the stack upward**.

---

2. Panic Propagation Example

```
package main


import "fmt"


func C() {
	panic("something went wrong")
}


func B() {
	fmt.Println("B start")
```

```
	C()
	fmt.Println("B end")
}

func A() {
	fmt.Println("A start")
	B()
	fmt.Println("A end")
}

func main() {
	A()
}
```

Execution Flow

main()

↓

A()

↓

B()

↓

C()

↓

panic occurs

Stack Unwinding

Go begins **unwinding the stack**:

panic in C

↑

return to B (but B stops executing)

↑

return to A (A stops executing)

↑

return to main

↑

program crashes

Output

A start

B start

panic: something went wrong

B end and A end never run.

---

3. Deferred Functions Run During Panic

Before leaving each function, **deferred functions execute**.

Example:

```go
package main

import "fmt"

func C() {
	defer fmt.Println("C defer")
	panic("panic in C")
}

func B() {
	defer fmt.Println("B defer")
	C()
}

func A() {
	defer fmt.Println("A defer")
	B()
}

func main() {
	A()
}
```

Output

C defer

B defer

A defer

panic: panic in C

Important Rule

Defer executes in **LIFO order (Last In First Out)**.

---

4. Panic Recovery

If a recover() exists in a deferred function, panic **stops propagating**.

Example:

```
package main


import "fmt"


func C() {
	panic("panic in C")
}


func B() {
	defer func() {
		if r := recover(); r != nil {
			fmt.Println("Recovered in B:", r)
```

```
            }
        }()
        C()
}

func A() {
        B()
        fmt.Println("A continues")
}

func main() {
        A()
}
```

Output

Recovered in B: panic in C

A continues

Propagation Stops Here

C panic

↑

B recover() catches panic

↑

stack unwinding stops

Program continues normally.

---

5. Panic Propagation Diagram

Without Recover

C() panic

↑

B() unwinds

↑

A() unwinds

↑

main() unwinds

↑

program crash

With Recover in B

C() panic

↑

B() recover() catches panic

↑

B returns normally

↑

A continues

↑

main continues

---

6. Important Rules

Rule 1

recover() **only works inside a deferred function**.

❌ Wrong

```
recover()
```

✓ Correct

```
defer func() {
   recover()
}()
```

Rule 2

recover() **only catches panic in the same goroutine**.

Panics in another goroutine cannot be recovered.

Rule 3

If panic reaches main() without recover → program terminates.

---

7. Real-World Use Case

Frameworks often use panic recovery to prevent server crashes.

Example pattern:

```
func middleware(handler http.HandlerFunc) http.HandlerFunc {
```

```
        return func(w http.ResponseWriter, r *http.Request) {

                defer func() {
                        if err := recover(); err != nil {
                                fmt.Println("Server recovered:", err)
                        }
                }()

                handler(w, r)
        }
}
```

This prevents a **single request panic from crashing the whole server**.

---

8. Panic vs Error (Very Important)

Situation	Use
expected failure	error
programmer bug	panic
corrupted state	panic
recoverable condition	error

Example:

file not found → error

nil pointer dereference → panic

---

9. Visual Summary

Function Call Stack

```
main
 │
 ▼
A
 │
 ▼
B
 │
 ▼
C → panic
 ↑
 ↑ stack unwinding
 ↑
recover() stops propagation
```

✅ **Key Idea**

panic → stack unwinding

defer executes

recover() may stop panic

otherwise program crashes